

SEQUENTIAL SYSTEMS & MICROPROCESSORS

B.E. Electrical Engineering · 2nd Year · 2nd Semester Examination, 2024

TIME: 3 HOURS FULL MARKS: 100 EACH PART: 50

PART I – SEQUENTIAL SYSTEMS

50 marks

Q1 Answer *any Two* of the following (compulsory; 2 × 5)

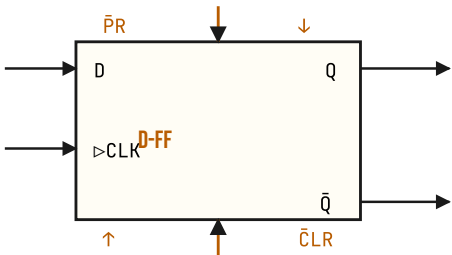
(A) PRESET AND CLEAR OPERATIONS OF D FLIP-FLOP

5

The PRESET ($\overline{PR}$) and CLEAR ($\overline{CLR}$) inputs are **asynchronous, direct** inputs that override the synchronous D input and the clock. They are typically active-low.

$\overline{PR}$	$\overline{CLR}$	ACTION
0	1	$Q \leftarrow 1$ (preset, force-set)
1	0	$Q \leftarrow 0$ (clear, force-reset)
1	1	Normal synchronous operation: $Q^+ = D$ on clock edge
0	0	Forbidden (Q and $\bar{Q}$ both = 1)

PRESET is used to initialise a register to a known non-zero state (e.g. ones-complement counter starting at 11 ... 1) while CLEAR is used to reset all flip-flops at power-up or upon a system-reset signal. Because they are independent of CLK, the flip-flop responds to them immediately rather than waiting for the next active edge.



D-FF with asynchronous active-low PRESET and CLEAR inputs.

(B) FICTITIOUS FLIP-FLOP WITH INPUTS A, B

Specification: $AB = 00 \Rightarrow Q^+ = 0$, $AB = 11 \Rightarrow Q^+ = 1$, $AB = 01 \Rightarrow Q^+ = Q$ (hold),
 $AB = 10 \Rightarrow Q^+ = \bar{Q}$ (toggle).

TRUTH TABLE

A	B	Q	Q ⁺	OPERATION
0	0	0	0	Reset
0	0	1	0	
0	1	0	0	Hold
0	1	1	1	
1	0	0	1	Toggle
1	0	1	0	
1	1	0	1	Set
1	1	1	1	

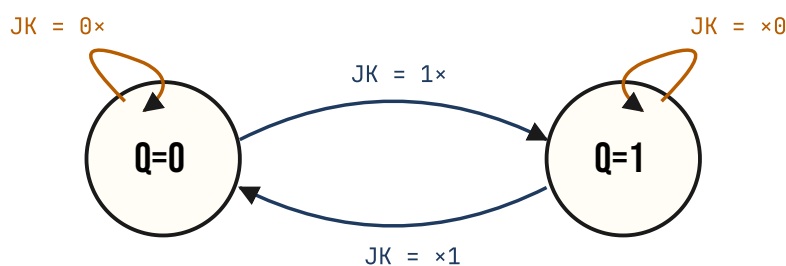
EXCITATION TABLE

Q	Q ⁺	A	B
0	0	0	x
0	1	1	x
1	0	x	0
1	1	x	1

Characteristic equation: $Q^+ = A\bar{Q} + BQ$, which can be recognised as the equation of a JK flip-flop with $J \equiv A$, $K \equiv \bar{B}$.

(C) STATE DIAGRAM OF JK FLIP-FLOP FROM ITS EXCITATION TABLE

Q	Q ⁺	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0



JK FF state diagram: hold, set, reset, toggle correspond to JK = 00, 10, 01, 11.

(D) SYNCHRONOUS VS ASYNCHRONOUS COUNTERS

5

A synchronous counter has all flip-flops driven by a common clock, with combinational logic generating the next-state inputs from the present state. All FFs change state simultaneously on the active edge, eliminating ripple delays and producing glitch-free outputs at the cost of more combinational hardware. An asynchronous (ripple) counter, in contrast, has only its least-significant flip-flop clocked by the system clock; each successive stage is clocked by the output of the previous stage. The propagation delay of every flip-flop accumulates, so the maximum operating frequency of an n-stage ripple counter is set by $f_{max} = 1/(n t_{pd})$ and the output may pass through transient invalid states. Synchronous counters are preferred for high-speed and decoder-driven applications; asynchronous counters are simpler and adequate for low-frequency uses such as time-keeping.

Q2

Excitation Table; SR → JK; Mod-3 counter analysis & Mod-6 extension

8 + 12

(A) EXCITATION TABLE; SR-FF → JK-FF CONVERSION

2 + 6

The excitation table of a flip-flop specifies, for every required transition $Q \rightarrow Q^+$, the input combination(s) that will produce it. It is the synthesis-oriented inverse of the truth table.

SR EXCITATION TABLE

Q	Q ⁺	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

SR → JK CONVERSION

Treat J and K as the new external inputs; for each (Q, J, K) read Q^+ from the JK truth table, then look up the required S, R in the SR excitation table:

Q	J	K	Q^+	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

K-map simplification yields:

$$S = J\bar{Q}, \quad R = KQ$$

Implementation: an SR flip-flop preceded by two AND gates — one forming $J \cdot \bar{Q}$ feeding S, and one forming $K \cdot Q$ feeding R — emulates a JK FF.

(B) VERIFY MOD-3 BEHAVIOUR OF FIG. Q2(B); EXTEND TO MOD-6

6 + 6

Both negative-edge JK FFs share the system clock. Reading the figure:

$$J_A = \bar{B}, \quad K_A = 1, \quad J_B = A, \quad K_B = 1.$$

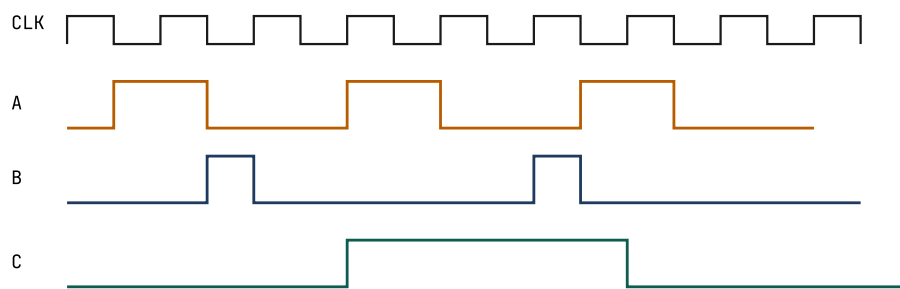
PULSE	A	B	J_A	K_A	J_B	K_B	A^+	B^+
0	0	0	1	1	0	1	1	0
1	1	0	1	1	1	1	0	1
2	0	1	0	1	0	1	0	0
3	0	0	1	1	0	1	1	0

The reachable cycle is $(A, B) = (0, 0) \rightarrow (1, 0) \rightarrow (0, 1) \rightarrow (0, 0)$ — three states, period 3. Hence the circuit is a **Mod-3 counter**. The unused state $(1, 1)$ is self-correcting: with $J_A = 0, K_A = 1$ and $J_B = 1, K_B = 1$ it forces the next state to $(0, 0)$.

MOD-6 EXTENSION

Connect output B to the clock input of a third JK FF (call it C) with $J_C = K_C = +V_{CC}$, so that C toggles on every **falling edge** of B. Because B has period 3, C produces a square wave of period 6. The complete state sequence becomes:

N	C	B	A	N	C	B	A
0	0	0	0	3	1	0	0
1	0	0	1	4	1	0	1
2	0	1	0	5	1	1	0



Q3

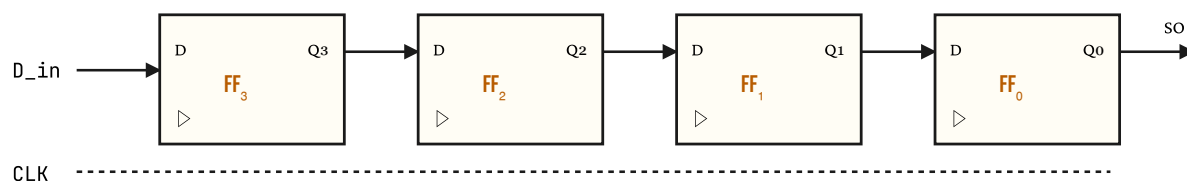
4-bit Shift-Right Register; Mod-4 irregular counter

12 + 8

(A) 4-BIT SHIFT-RIGHT REGISTER WITH D FFS AND SERIAL ENTRY OF "0100"

4 + 8

Four D flip-flops are cascaded: $D_3 \leftarrow D_{in}$, $D_2 \leftarrow Q_3$, $D_1 \leftarrow Q_2$, $D_0 \leftarrow Q_1$, all clocked from the same CLK. The register shifts right on every active edge.



Block diagram of a 4-bit Serial-In Serial-Out shift-right register using D flip-flops.

To enter "0100" serially with the LSB applied first, the bit stream into D_{in} is 0, 0, 1, 0 over four successive clock periods. The register contents evolve as follows:

PULSE	D_{IN}	Q_3	Q_2	Q_1	Q_0
Initial	—	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	1	1	0	0	0
4	0	0	1	0	0

After four clock pulses the register holds $Q_3Q_2Q_1Q_0 = 0100$.

(B) MOD-4 IRREGULAR COUNTER 00 $\rightarrow$ 10 $\rightarrow$ 11 $\rightarrow$ 01 $\rightarrow$ 00 USING D FFS

8

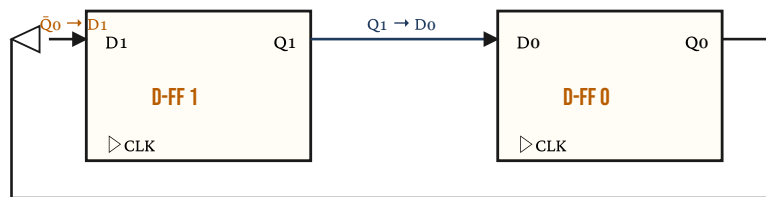
Let Q_1Q_0 denote the present state. From the desired sequence:

Q_1	Q_0	Q_1^+	Q_0^+	D_1	D_0
0	0	1	0	1	0
1	0	1	1	1	1
1	1	0	1	0	1
0	1	0	0	0	0

K-map simplification (with no don't-cares since all four states are used):

$$D_1 = \bar{Q}_0, \quad D_0 = Q_1$$

Implementation: D_1 is driven by an inverter from Q_0 (or simply by $\bar{Q}_0$ from the FF), and D_0 is wired directly from Q_1 . The two FFs share the clock.



Mod-4 irregular counter using D FFs: $D_1 = \bar{Q}_0$, $D_0 = Q_1$.

Q4

Mealy vs Moore; synthesis from timing diagram

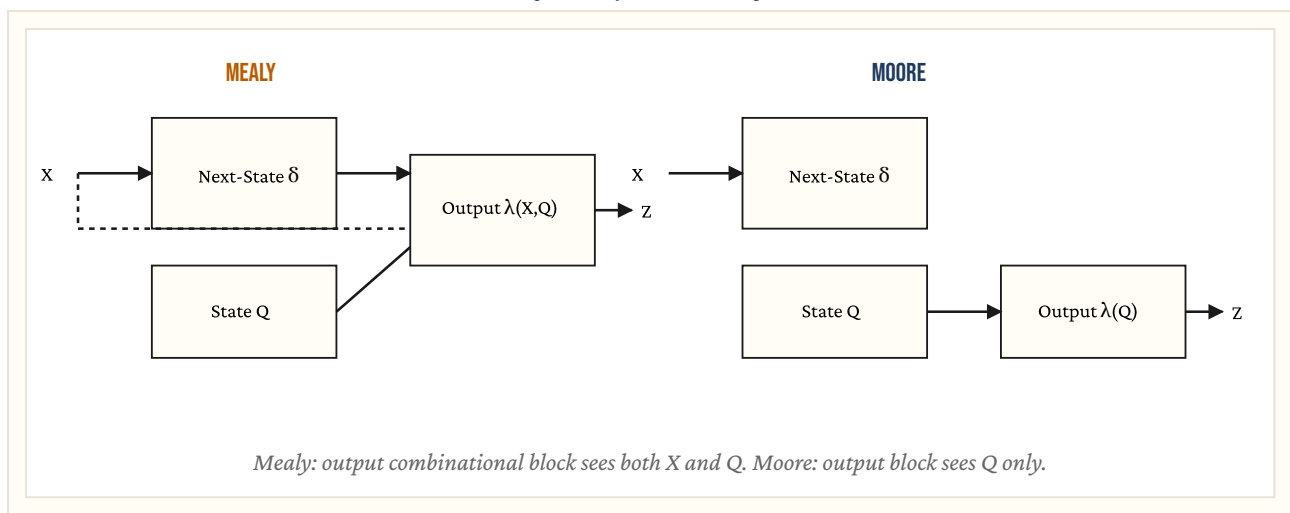
6 + 14

(A) MEALY VS MOORE MODELS – SCHEMATIC DIFFERENCE

6

A finite state machine (FSM) consists of a state register Q , a next-state combinational block $\delta(X, Q)$, and an output combinational block λ . The two canonical models differ in the inputs to the output block.

In the **Mealy** model the output depends on both the present state and the present input: $Z = \lambda(X, Q)$. The output therefore changes as soon as the input changes, even between clock edges, and may show transient glitches. In the **Moore** model the output depends only on the present state: $Z = \lambda(Q)$. The output changes only when the state changes, i.e. on a clock edge, giving glitch-free behaviour but typically requiring more states. A Mealy machine is usually more compact (fewer flip-flops); a Moore machine is easier to interface to other synchronous circuits.



(B) SYNCHRONOUS CIRCUIT SYNTHESIS FROM GIVEN TIMING DIAGRAM

4 + 4 + 6

Reading the waveforms of CLK, X , Y and Z from Fig. Q4(b) with the convention that on each rising clock edge we record (X, Y, Z) :

The output Z is non-zero only when both X and Y are high **and** the previous sample also had $X \cdot Y$ high — i.e. the FSM remembers whether the immediately preceding input pair was (1,1). Two states suffice: S_0 (last input was not 11) and S_1 (last input was 11). With Mealy output:

(I) STATE DIAGRAM

$S_0 \xrightarrow{XY=11/0} S_1, S_1 \xrightarrow{XY=11/1} S_1$, both states return to S_0 on $XY \neq 11$ with $Z = 0$.

(II) STATE SYNTHESIS TABLE (WITH Q ENCODING $S_0 = 0, S_1 = 1$)

Q	X	Y	$Q^+ = D$	Z
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = XY, \quad Z = XYQ$$

(III) REALISATION WITH ONE D FLIP-FLOP

An AND gate produces $D = XY$ and another AND of three inputs produces $Z = XYQ$. The single D-FF stores the "last input was 11" indicator.

(A) UNIVERSAL SHIFT-REGISTER – DEFINITION AND 4-BIT MODES

2 + 8

A universal shift register can perform all four standard register operations — parallel load, shift right, shift left, and hold — under software control of two mode-select lines. Each stage is a D flip-flop preceded by a 4-to-1 multiplexer whose four inputs are: the FF's own output (hold), the right-neighbour's output (shift right), the left-neighbour's output (shift left), and an external parallel-input bit. The same select lines $s_1 s_0$ are wired to every stage, so the entire register selects one mode per clock cycle.

S_1	S_0	MODE	MUX SELECTS
0	0	Hold	Own Q_i
0	1	Shift right	Q_{i+1}
1	0	Shift left	Q_{i-1}
1	1	Parallel load	External I_i

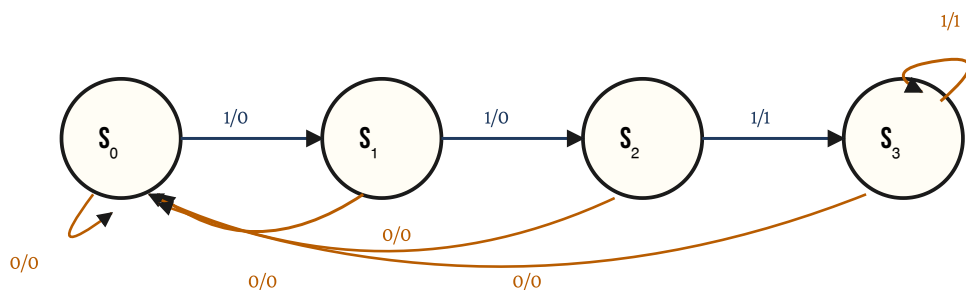
The 74LS194 is the classical TTL universal shift register implementing this scheme. Two extra serial-input pins (SR-SI and SL-SI) allow chaining for longer registers.

(B) SEQUENCE DETECTOR – THREE OR MORE CONSECUTIVE 1s

4 + 6

(I) STATE TRANSITION DIAGRAM (MEALY)

Four states are sufficient: S_0 (no 1s yet), S_1 (one 1), S_2 (two 1s), S_3 (three or more 1s). Output goes high on every additional 1 once the third has been seen.



Mealy machine: detects three or more consecutive 1s, output asserted from the third 1 onwards.

(II) STATE TABLE AND D-FF REALISATION

State assignment $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$:

Q_1	Q_0	X	D_1	D_0	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	1	1	1

$$D_1 = x(Q_0 + Q_1), \quad D_0 = x(\bar{Q}_1\bar{Q}_0 + Q_1), \quad z = xQ_1$$

The detector is built from two D flip-flops, three AND gates, two OR gates and one inverter; clock and asynchronous reset are common.

PART II – 8085 MICROPROCESSOR

50 marks

GROUP A – ANSWER ANY FOUR ($4 \times 5 = 20$)

A.1

Why de-multiplexing of address and data bus is required

5

The 8085 packages the lower address byte A_7-A_0 and the bidirectional data byte D_7-D_0 on the same eight pins AD_7-AD_0 to economise on pin count. External memory and I/O devices, however, require a stable address throughout the read or write cycle while data appears on the bus only for a few T-states. De-multiplexing separates these two functions: an external 8-bit transparent latch (typically 74LS373) is enabled by the ALE pulse — which goes high only during T_1 when valid address is on the bus — so that on the falling edge of ALE the lower address byte is captured and held for the rest of the machine cycle. The data lines AD_7-AD_0 then carry data alone during T_2, T_3 , while the latched A_7-A_0 remains stable for the chip-select and storage-array decoders. Without this separation, address and data would interfere and no reliable memory access would be possible.

A.2

Program Counter vs Stack Pointer

5

Both PC and SP are 16-bit registers in the 8085 but they serve very different roles. The Program Counter holds the address of the **next instruction byte** to be fetched from memory; it is auto-incremented after every fetch and is overwritten by the target during JMP, CALL and RET, and by the vector during interrupts. The Stack Pointer holds the address of the **top element** of a LIFO stack maintained somewhere in user RAM. SP is decremented by 2 before every PUSH or implicit save (CALL, interrupt response) and incremented by 2 after every POP or RET. PC tracks program flow and is read-only to the programmer except through control-transfer instructions; SP is read-write through LXI SP, SPHL and DAD SP. Errors in PC manipulation crash the program; errors in SP corrupt the stack and cause subroutines to return to wrong addresses.

A.3

Wait state and wait-state generator

5

A wait state T_W is an additional clock period inserted by the CPU into a memory or I/O machine cycle so that a slow device has enough time to complete the transfer. The CPU samples its READY input on the rising edge of

T_2 ; if READY is low, T_W is inserted (and another READY sample is taken at the end of T_W) until READY returns high, after which T_3 proceeds normally.

A typical wait-state generator uses a chain of D flip-flops or a shift register clocked by the system clock and triggered by the device chip-select strobe. Its output gates the READY line low for the desired number of T-states. The simplest version uses a single D-FF: when chip-select goes active the FF output goes low for one T period, asserting one wait state, then auto-clears.

A.4

Image addresses in partial decoding

5

Partial address decoding uses fewer than all the upper address lines of the CPU to select a memory chip. Any upper address line not connected to the decoder becomes a "don't care" bit, with the result that the chip is selected at multiple non-overlapping ranges in the memory map. Each such alias is called an **image (or shadow) address**.

For example, suppose a 2 KB RAM is selected by $\overline{CS} = A_{15}\overline{A}_{14}$, with A_{13} and A_{12} left unconnected. The chip's 11 internal lines $A_{10}-A_0$ select the byte. The four base ranges are 8000H–87FFH, 8800H–8FFFH, 9000H–97FFH, 9800H–9FFFH — four images of the same 2 KB. Image addresses simplify the decoder hardware but waste address space and may cause unintended writes if other devices are mapped in those reserved regions. Image addresses are eliminated by routing every unused upper line into the decoder so that only one unique range selects each chip — this is called **absolute (or full) decoding**.

A.5

Instruction cycle, machine cycle, T-state

5

An **instruction cycle** is the total time required for the CPU to fetch, decode and execute one complete instruction. It is composed of one or more machine cycles. A **machine cycle** is the time required for the CPU to perform one external operation on the buses — such as opcode fetch, memory read, memory write, I/O read, I/O write or interrupt acknowledge — and lasts 3 to 6 T-states. A **T-state** (timing state) is one period of the system clock; it is the smallest unit of CPU activity and equals the inverse of the system clock frequency. With a 3 MHz system clock, $T = 0.333 \mu s$. The 8085 has instructions ranging from 4 T-states (e.g. MOV r,r') to 18 T-states (CALL).

A.6

Memory-mapped I/O vs I/O-mapped I/O

5

In **memory-mapped I/O**, peripheral registers are decoded as if they were memory locations and are accessed using the full memory-reference instructions (LDA, STA, MOV M, etc.) with the $\overline{IO/\overline{M}}$ line low. The advantage is the entire 8085 instruction set is available for I/O, including arithmetic and logical operations on port data. The cost is that part of the 64 KB memory map is consumed by I/O addresses.

In **I/O-mapped I/O** (also called isolated I/O), peripherals occupy a separate 256-byte address space accessed only by IN and OUT instructions, with $\text{IO}/\bar{\text{M}}$ high. The 16-bit port number is duplicated on $A_{15}\text{--}A_8$ and $A_7\text{--}A_0$ during the I/O machine cycle. Memory and I/O spaces are disjoint, so no memory addresses are wasted, but only IN/OUT can move data between A and a port; arithmetic on port data requires a copy through the accumulator.

Example: reading a status register at port 21H. Memory-mapped (suppose mapped at 8021H): `LDA 8021H` .
I/O-mapped: `IN 21H` .

A.7

ISR vs Subroutine

5

A subroutine is a block of code executed in response to a CALL instruction in the main program; the programmer fully controls when it runs. The CPU pushes only the return address onto the stack, executes the subroutine, and returns via RET. An **Interrupt Service Routine (ISR)**, in contrast, is invoked asynchronously by an external (or internal) hardware event signalled on TRAP, RST 5.5/6.5/7.5 or INTR. The CPU completes its current instruction, disables further maskable interrupts (clears IE), pushes the PC onto the stack, vectors to the predefined ISR address, executes the routine, and on RET re-enables interrupts (if EI is the last instruction). Because the interruption can occur at any point, an ISR must additionally save and restore any registers it modifies (PUSH PSW, PUSH H, etc.) so the interrupted program resumes transparently.

Example: a subroutine is called by the programmer to compute a checksum (CALL CKSUM); an ISR is invoked when a keyboard scan completes and the keyboard controller raises RST 6.5.

A.8

RIM to sense pending interrupts

5

RIM (Read Interrupt Mask) loads the accumulator with an 8-bit status word describing the interrupt subsystem. The bit pattern is

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
SID	P7.5	P6.5	P5.5	IE	M7.5	M6.5	M5.5

where bits D₆–D₄ are the **pending** indicators for RST 7.5, 6.5 and 5.5 (a 1 means the interrupt is pending but masked or held for service), bit D₃ is the master interrupt-enable flag, bits D₂–D₀ are the current mask states, and bit D₇ is the serial input-data line. To check whether RST 6.5 is currently pending while masked, the typical sequence is

```
RIM
ANI 20H ; isolate D5 (P6.5)
JNZ P65_PENDING
```

Hence RIM lets software poll for events that were generated while the interrupt was disabled or masked.

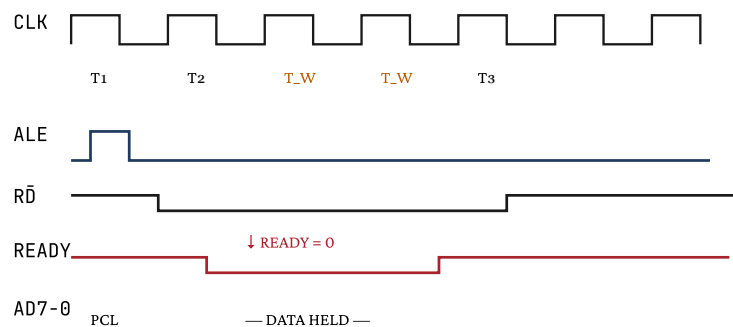
GROUP B — ANSWER ANY THREE ($3 \times 5 = 15$)

B.1

Memory-read timing diagram, with READY low at mid-T₂

5

A normal memory-read machine cycle has three T-states. During T_1 the CPU drives the address onto AD and $A_{15}-A_8$ and asserts ALE ; in T_2 the address is latched, $\overline{RD}$ goes low and $IO/\overline{M}$ stays low; in T_3 the data on AD_7-AD_0 is read into the CPU and $\overline{RD}$ goes high. If $READY$ is sampled low at the rising edge of T_2 , the CPU inserts wait states T_W until $READY$ is sampled high again, then proceeds to T_3 . Address, $\overline{RD}$, ALE and $IO/\overline{M}$ all remain held throughout the wait states.

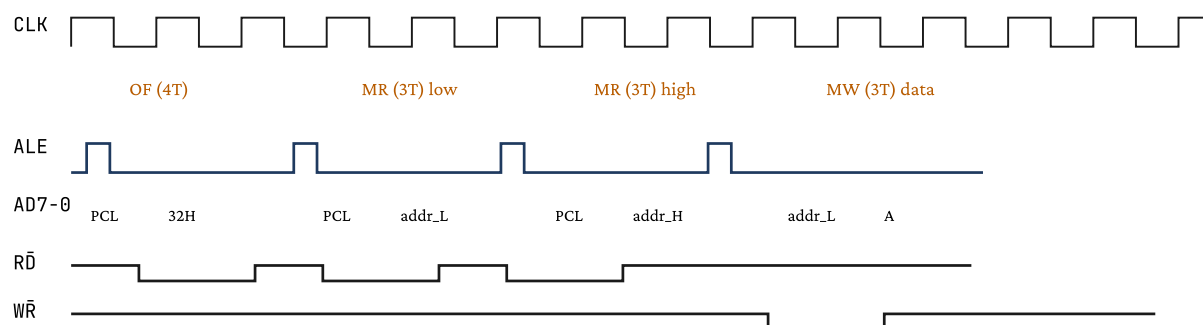


B.2

Timing diagram of STA <16-bit> (32H), 3 machine cycles, 10 T-states

5

Note: STA actually takes 13 T-states ($4+3+3+3$); the question's 10-T figure is interpreted as the three principal cycles after opcode fetch. Three machine cycles in the read-and-store phase: OF (4T) → MR (3T) → MR (3T) → MW (3T). The two memory reads bring in the low and high bytes of the destination address; the memory write puts A into that location.



B.3

Vectored interrupts; calculation of ISR address

5

A vectored interrupt is one whose service-routine address is pre-defined by the CPU rather than supplied by the interrupting device. The 8085 has four vectored interrupts: TRAP, RST 5.5, RST 6.5, RST 7.5. When the CPU acknowledges any of them it pushes the PC onto the stack and loads the PC with the corresponding vector address.

The vector address for an RST $n.5$ interrupt is computed as $\text{vector} = n.5 \times 8 = 8n + 4$, expressed in hex:

INTERRUPT	TRIGGER	VECTOR ADDRESS
TRAP	Edge + level (NMI)	0024H
RST 5.5	Level-sensitive	002CH
RST 6.5	Level-sensitive	0034H
RST 7.5	Edge-triggered	003CH

Example calculation for RST 7.5: $7.5 \times 8 = 60_{10} = 003CH$. The eight bytes from 003CH onward are typically reserved for a JMP to the actual service routine since they are too short for full code.

B.4

Delay calculation: 4 MHz crystal & given subroutine

5

System clock $f_{sys} = 4/2 = 2 \text{ MHz}$, $T = 0.5 \mu s$.

```

    LXI B, 0FFFFH    ; 10 T-states (executed once)
BACK: DCX B          ; 6 T-states
    MOV A, C          ; 4 T-states
    ORA B              ; 4 T-states
    JNZ BACK          ; 10 (taken) / 7 (last)

```

BC starts at FFFFH = 65535. The loop body (DCX through JNZ taken) costs $6 + 4 + 4 + 10 = 24$ T-states per pass. The loop runs 65535 passes; the final pass takes $6 + 4 + 4 + 7 = 21$ T-states.

$$T_{loop} = 65534 \times 24 + 21 = 1,572,816 + 21 = 1,572,837 \text{ T-states}$$

$$T_{total} = 10 + 1,572,837 = 1,572,847 \text{ T-states}$$

$$\Delta t = 1,572,847 \times 0.5 \mu s \approx 786.4 \text{ ms}$$

B.5

Interface 2 KB RAM at 2000H, 4000H and I/O at 17H, 07H — partial decoding

5

For a 2 KB RAM the chip uses internal lines A_{10} – A_0 . To place one chip at 2000H–27FFH and another at 4000H–47FFH, the chip-select for each can be derived from a few of the upper address lines. Using partial decoding (some lines unconnected) gives:

DEVICE	BASE	CHIP-SELECT CONDITION
RAM ₁	2000H	$A_{13}\bar{A}_{14}\bar{A}_{15} = 1$ (using only A_{13} – A_{15})

RAM ₂	4000H	$\bar{A}_{13}A_{14}\bar{A}_{15} = 1$
I/O port (17H)	17H	$A_4A_2A_1A_0 = 1$ during I/O cycle (unused lines = images)
I/O port (07H)	07H	$A_2A_1A_0 = 1$ during I/O cycle

The two RAMs share data lines D_7-D_0 , $\overline{RD}$ and $\overline{WR}$. Each chip-select is generated by a 3-input gate fed from the relevant A_{15} , A_{14} , A_{13} lines, and the lower 11 lines are common. The I/O ports use a 3-to-8 decoder driven from the lower address byte $A_2A_1A_0$ during the I/O cycle, with $IO/\bar{M}$ as enable. Because higher I/O address lines are unconnected, every port has multiple image addresses inside the 256-port space.

B.6

Control word format of 8255 PPI in BSR mode

5

The 8255 Programmable Peripheral Interface has two control-word formats distinguished by bit D₇. With D₇ = 0 the chip is in **Bit-Set/Reset (BSR) mode**, used to set or clear individual bits of port C without disturbing the others. The format is

D ₇	D ₆ -D ₄	D ₃ -D ₁	D ₀
0	X (don't care)	Bit select (PC0-PC7)	1 = set, 0 = reset

Example — set PC5 to 1 with the 8255 control register at port 23H:

```
MVI A, 0BH      ; 0000 1011 → BSR, bit 5 (101), set
OUT 23H         ; write to control register
```

To reset PC5 instead, use 0AH (D₀ = 0). BSR mode is particularly useful for driving control lines, LEDs, relays, or generating handshake signals without read-modify-write of the entire port.

B.7

Interrupt priority & masking of vector interrupts

5

The 8085 implements a fixed hardware priority among its five interrupt inputs:

PRIORITY	INTERRUPT	MASKABLE?	VECTOR
1 (highest)	TRAP	No	0024H
2	RST 7.5	Yes (SIM)	003CH
3	RST 6.5	Yes (SIM)	0034H
4	RST 5.5	Yes (SIM)	002CH
5 (lowest)	INTR	Yes (EI/DI)	Supplied by device

The vectored RST 5.5, 6.5 and 7.5 interrupts are individually maskable through the SIM instruction. SIM writes the lower five bits of A into the interrupt-mask register: bits M5.5, M6.5, M7.5 (1 = masked); MSE = 1 enables those mask bits to take effect; R7.5 = 1 resets the RST 7.5 internal flip-flop. INTR is enabled by EI and disabled by DI globally; TRAP is non-maskable. Thus a typical pattern that masks RST 6.5 alone is

```

EI
MVI A, 0AH      ; 0000 1010 → MSE=1, M6.5=1
SIM

```

GROUP C — ANSWER ANY THREE (3 × 5 = 15)

C.1
Separate even and odd numbers from an array of N elements
5

```

; Source array at 8000H, length N at 9000H
; Even array → 8100H, Odd array → 8200H
LDA 9000H
MOV C, A      ; C = N (loop counter)
LXI H, 8000H  ; source pointer
LXI D, 8100H  ; even pointer
LXI SP, 87FFH ; (only if needed)
NEXT: MOV A, M
RAR          ; LSB → CY
JC ODD
MOV A, M     ; restore
STAX D       ; store in even array
INX D
JMP CONT
ODD:  PUSH H      ; save src ptr
LXI H, 8200H
MOV A, M     ; cannot — use a different ptr
POP H
; cleaner: keep odd-pointer in BC if available
CONT: INX H
DCR C
JNZ NEXT
HLT

```

(A cleaner version uses BC for the source counter, HL for the source array, DE for one destination pointer and a register-pair save/restore for the second pointer.) The test of the LSB by RAR places the data's bit 0 into CY; if CY = 1 the number is odd, otherwise even.

C.2
Sort N elements ascending; count repeated numbers
5

Bubble sort with adjacent-pair compare-exchange, plus a final pass that counts repetitions in the now-sorted array.

```

; Array at 8000H, length N at 9000H, repeat count at 9001H
LDA 9000H

```



```

MOV B, A          ; outer counter
DCR B             ; N-1 passes
PASS: LXI H, 8000H
MOV C, B
CMP1: MOV A, M
INX H
CMP M
JC SKIP           ; if A < M no swap
JZ SKIP           ; equal — no swap
MOV D, M
MOV M, A
DCX H
MOV M, D
INX H
SKIP: DCR C
JNZ CMP1
DCR B
JNZ PASS
; -- count repeats --
LDA 9000H
MOV C, A
DCR C
LXI H, 8000H
MVI E, 00H
RPT: MOV A, M
INX H
CMP M
JNZ NXT
INR E
NXT: DCR C
JNZ RPT
MOV A, E
STA 9001H
HLT

```

C.3

Factorial of an 8-bit number — flow chart and program

5

Algorithm: read n from memory; initialise result to 1; multiply result by $n, n - 1, \dots, 2$ using repeated addition because the 8085 has no multiply instruction; store result.

```

; n at 8000H, n! (16-bit) at 8050H-8051H
LDA 8000H
MOV C, A          ; C = n (outer counter)
LXI H, 0001H      ; HL = running product
FACT: MOV A, C
CPI 01H
JZ DONE           ; 0! = 1! = 1
MOV D, H
MOV E, L          ; DE = previous product
MOV B, C

```

```

        DCR B           ; multiply by C: add (C-1) more times
MUL:    DAD D
        DCR B
        JNZ MUL
        DCR C
        JMP FACT
DONE:   SHLD 8050H
        HLT

```

Flow-chart skeleton: *Start* → load n → if $n \leq 1$ result = 1 → else loop: result ← result + result + ... ($n - 1$ times) using DAD, decrement n , repeat → store → halt.

C.4

Square root of a number

5

Use the well-known property $\sum_{i=1}^n (2i - 1) = n^2$. Subtract successive odd numbers (1, 3, 5, ...) from the input until the remainder becomes zero or negative; the count of subtractions is the integer square root.

```

        ; Input at 8000H, √ stored at 8050H
LDA 8000H
MVI B, 00H           ; counter (= √)
MVI C, 01H           ; current odd number
LOOP:  SUB C
        JC END        ; went negative ⇒ done
        INR B
        JZ END        ; exact square
        INR C
        INR C           ; next odd: 1, 3, 5, ...
        JMP LOOP
END:   MOV A, B
        STA 8050H
        HLT

```

For input 36 → subtract 1, 3, 5, 7, 9, 11 → six subtractions before remainder zero → result = 6.

C.5

Convert 2-digit BCD to 8-bit binary — flow chart & program

5

For a packed 2-digit BCD number D_1D_0 stored in one byte (high nibble = tens, low nibble = units), the binary equivalent is $D_1 \times 10 + D_0$. Multiplication by 10 is achieved as $\text{tens} \times 10 = \text{tens} \ll 3 + \text{tens} \ll 1$ or by repeated addition.

```

        ; BCD byte at 8000H, binary result at 8001H
LDA 8000H
MOV B, A
ANI 0FH           ; A = units
MOV C, A

```

```

MOV  A, B
ANI  F0H
RRC
RRC
RRC
RRC          ; A = tens
MOV  D, A
MVI  E, 0AH      ; multiplier 10
MVI  A, 00H
MUL10: ADD  D
DCR  E
JNZ  MUL10      ; A = tens × 10
ADD  C          ; + units
STA  8001H
HLT

```

Flow chart: input → mask low nibble → save units → mask high nibble, rotate right 4 → save tens → multiply tens by 10 (repeated addition) → add units → store → halt.

C.6

1 ms pulse on port 01H, started by RST 7.5 button

5

Configure the system clock at the standard 2 MHz ($T = 0.5 \mu s$). One ms is therefore 2000 T-states. A simple delay loop with DCR + JNZ uses 14 T per iteration, so a count of $\lceil 2000/14 \rceil = 143 \approx 8FH$ gives roughly 1 ms.

```

; Main: enable RST 7.5, then idle
LXI SP, 27FFH
EI
MVI A, 0BH      ; MSE=1, M5.5=1, M6.5=1, M7.5=0 ⇒ enable RST 7.5 only
SIM
IDLE: JMP  IDLE

; ISR at vector 003CH
ORG  003CH
JMP  PULSE
ORG  2000H
PULSE: PUSH PSW
      PUSH B
      MVI A, 01H
      OUT 01H      ; rising edge
      MVI B, 8FH   ; ≈ 1 ms delay
DLY:  DCR B
      JNZ DLY
      XRA A
      OUT 01H      ; falling edge
      POP B
      POP PSW
      EI
      RET

```

Each press of the push-button pulls the RST 7.5 input high; the internal flip-flop latches the request and on completion of the current instruction the CPU vectors to 003CH. The ISR drives port 01H high, delays for ~1 ms, drops it low, re-enables interrupts and returns.

— END OF 2024 SOLUTIONS —